

Técnicas de Diseño de Algoritmos I

Algoritmos Voraces y Programación Dinámica

AGENDA DE LA CLASE

Contenido:

- Repaso: ¿Qué vimos en la clase anterior?
- Introducción a las técnicas de diseño
- Algoritmos Voraces (Greedy)
 - Principios fundamentales
 - Problema del cambio de monedas
 - Otros casos de aplicación
- Programación Dinámica
 - Concepto y filosofía
 - Subproblemas y optimalidad
 - Secuencia de Fibonacci
- Comparación entre ambas técnicas
- Actividades prácticas

Tarea para la próxima clase

TÉCNICAS DE DISEÑO - PANORAMA GENERAL

¿Por qué necesitamos diferentes técnicas?

Clasificación de técnicas de diseño:

- Fuerza bruta
- Divide y vencerás
- **Algoritmos voraces**
- **Programación dinámica**
- Backtracking

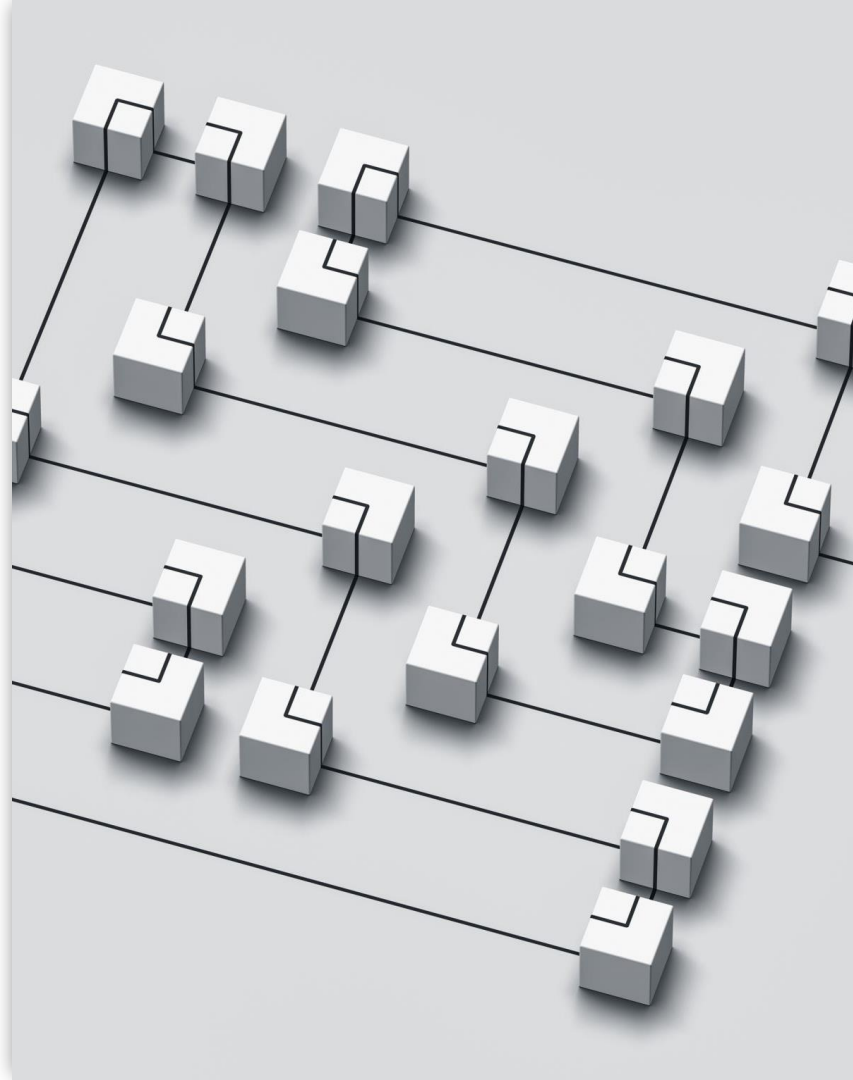
Criterios para elegir la técnica adecuada

Relación con la complejidad computacional

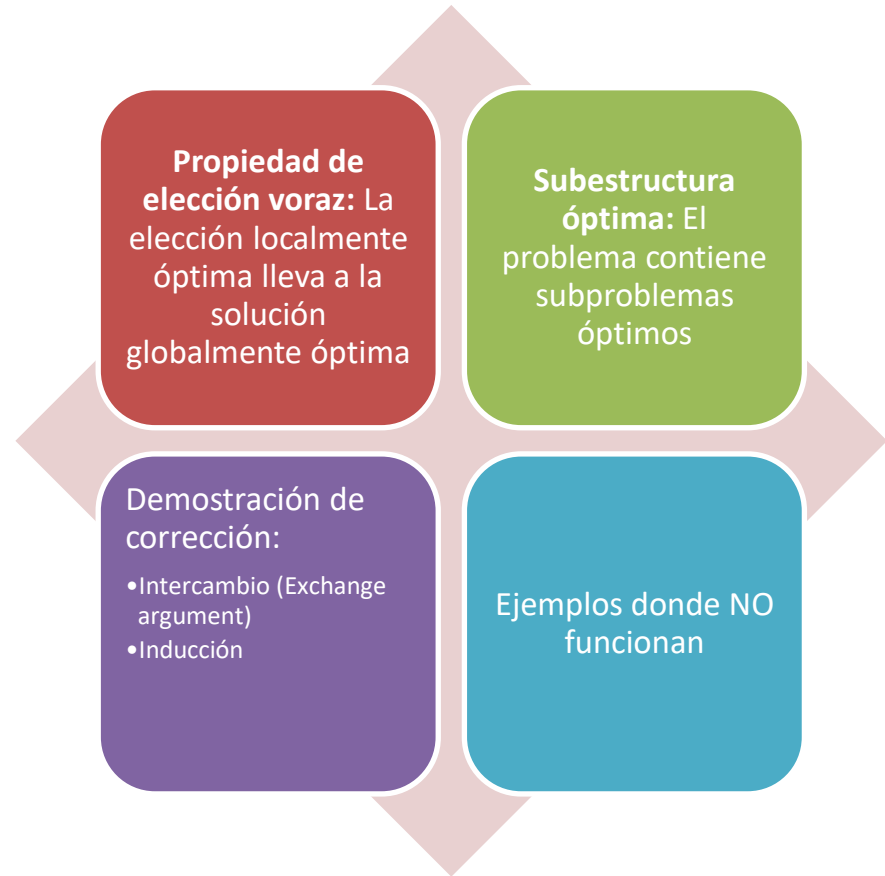
ALGORITMOS VORACES

- FILOSOFÍA

- Definición: "Tomar la mejor decisión local en cada paso"
- Características principales:
 - Decisiones irrevocables
 - Optimización local → ¿Optimización global?
 - Simplicidad de implementación
- Esquema general del algoritmo voraz
¿Cuándo funcionan y cuándo no?



PROPIEDADES DE LOS ALGORITMOS VORACES



PROBLEMA DEL CAMBIO DE MONEDAS

- **Enunciado:** Dado un monto y denominaciones de monedas, encontrar el menor número de monedas
- Sistema monetario argentino: \$1000, \$500, \$200, \$100, \$50, \$20, \$10, \$5, \$2, \$1
- **Estrategia voraz:** Siempre elegir la denominación más grande posible
- Ejemplo práctico: cambio para \$1847
- ¿Por qué funciona con nuestro sistema monetario?



IMPLEMENTACIÓN - CAMBIO DE MONEDAS

- Complejidad temporal:
 $O(n)$ donde n es
número de
denominaciones
- Complejidad espacial:
 $O(k)$ donde k es
número de monedas
en la solución

```
Algoritmo CambioMonedas(monto, denominaciones[])  
    resultado = []  
    para cada denominación d en denominaciones (ordenadas desc):  
        mientras monto >= d:  
            resultado.agregar(d)  
            monto = monto - d  
    retornar resultado
```

OTROS EJEMPLOS DE ALGORITMOS VORACES



Problema de la mochila fraccional

Maximizar valor llevando objetos fraccionables

Estrategia: ordenar por ratio valor/peso



Planificación de actividades

Seleccionar máximo número de actividades no superpuestas

Estrategia: ordenar por tiempo de finalización



Algoritmo de Huffman (códigos de compresión)



Algoritmos de grafos: Kruskal, Prim, Dijkstra



PROGRAMACIÓN DINÁMICA - FILOSOFÍA

- **Principio:** "Resolver problemas complejos descomponiéndolos en subproblemas más simples"
- Diferencia clave con voraces: **evaluamos todas las opciones**
- Almacenamos soluciones de subproblemas (memoización)
- **Bottom-up vs Top-down**
- Cuando los subproblemas se superponen

PROPIEDADES DE LA PROGRAMACIÓN DINÁMICA

Subestructura óptima: La solución óptima contiene soluciones óptimas de subproblemas

Subproblemas superpuestos: Los mismos subproblemas aparecen múltiples veces

Principio de Bellman: "Una política óptima tiene la propiedad de que, sin importar el estado y decisión inicial, las decisiones restantes constituyen una política óptima"

SECUENCIA DE FIBONACCI - PROBLEMA CLÁSICO

Definición: $F(n) = F(n-1) + F(n-2)$, con $F(0)=0$, $F(1)=1$

Enfoque ingenuo recursivo: Exponencial $O(2^n)$

Problema: Cálculos redundantes

Árbol de recursión para $F(5)$

¿Cuántas veces calculamos $F(2)$?

FIBONACCI CON PROGRAMACIÓN DINÁMICA

- **Enfoque Bottom-up (Tabulación):**

```
función FibTabla(n):  
    si n <= 1: retornar n  
    dp[0] = 0, dp[1] = 1  
    para i = 2 hasta n:  
        dp[i] = dp[i-1] + dp[i-2]  
    retornar dp[n]
```

- **Enfoque Top-down (Memoización):**

```
memo = []  
función FibMemo(n):  
    si memo[n] != -1: retornar memo[n]  
    si n <= 1: memo[n] = n  
    sino: memo[n] = FibMemo(n-1) + FibMemo(n-2)  
    retornar memo[n]
```

COMPARACIÓN DE ENFOQUES - FIBONACCI

Enfoque	Complejidad Tiempo	Complejidad Espacio	Observaciones
Recursivo	$O(2^n)$	$O(n)$	Inviabile para $n > 40$
Memoización	$O(n)$	$O(n)$	Top-down
Tabulación	$O(n)$	$O(n)$	Bottom-up
Optimizado	$O(n)$	$O(1)$	Solo últimos 2 valores

OTROS EJEMPLOS DE PROGRAMACIÓN DINÁMICA



Problema de la mochila 0/1

Objetos indivisibles,
maximizar valor



Longest Common Subsequence (LCS)

Comparación de
secuencias de
ADN/texto



Camino más corto (Floyd-Warshall)

Todos los pares de
vértices



Problemas de optimización en strings

Edit distance,
palindromes

VORACES vs PROGRAMACIÓN DINÁMICA

Aspecto	Algoritmos Voraces	Programación Dinámica
Estrategia	Decisión local óptima	Evalúa todas las opciones
Reversibilidad	Decisiones irrevocables	Puede reconsiderar
Complejidad	Generalmente menor	Generalmente mayor
Garantía	No siempre óptimo	Siempre óptimo (si aplica)
Memoria	Mínima	Puede requerir mucha
Casos de uso	Problemas con prop. voraz	Subproblemas superpuestos

¿CUÁNDO USAR CADA TÉCNICA?

Usar Algoritmos Voraces cuando:

- Existe propiedad de elección voraz
- Las decisiones locales llevan al óptimo global
- Se requiere eficiencia temporal
- Ejemplos: Huffman, MST, cambio de monedas

Usar Programación Dinámica cuando:

- Hay subproblemas superpuestos
- Se requiere solución óptima garantizada
- Los subproblemas tienen subestructura óptima
- Ejemplos: Mochila 0/1, LCS, Fibonacci

ACTIVIDAD GRUPAL

- **Ejercicio 1 (Grupos de 3):** Resolver el problema del cambio para €127 con monedas europeas (€50, €20, €10, €5, €2, €1)
- **Ejercicio 2 (Grupos de 3):** Calcular $F(10)$ manualmente usando:
 - Recursión simple (mostrar árbol)
 - Programación dinámica bottom-up
- **Tiempo:** 15 minutos
- **Presentación:** 2 grupos por ejercicio



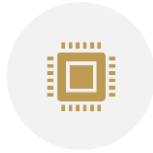
CASOS DE ESTUDIO EMPRESARIALES



Google: Algoritmos voraces en Ad placement y resource scheduling



Netflix: Programación dinámica para recomendaciones y optimización de streaming



Amazon: Ambas técnicas en logistics optimization y pricing algorithms



Mercado Libre: Optimización de envíos y gestión de inventario



Fintech argentinas: Algoritmos para scoring crediticio y detección de fraude

HERRAMIENTAS Y PROFILING

Python: time,
timeit,
memory_profiler

Haskell: criterion,
ghc-prof

Visualización:
matplotlib para
comparar
rendimiento

Debugging: Trace
de ejecución en
algoritmos
recursivos

Recursos online:
LeetCode,
HackerRank,
Codeforces

GitHub:
Repositorios con
implementaciones
de referencia



TAREA PARA LA PRÓXIMA CLASE

- **Lectura obligatoria:**
 - Cormen et al., Capítulos 15.1-15.3 y 16.1-16.2
 - Kleinberg & Tardos, Capítulos 4.1-4.3 y 6.1-6.3
- **Implementar:** Algoritmo de cambio de monedas en Python y Haskell
- **Resolver:** 3 problemas del TP1 (serán publicados en campus)
- **Preparar:** Dudas sobre divide y vencerás para próxima clase
- **Fecha límite:** Implementaciones hasta el miércoles 23:59

PRÓXIMA CLASE



**Tema: Técnicas de
Diseño de Algoritmos II**



Contenidos:

Divide y Vencerás: MergeSort,
QuickSort

Backtracking: N-Reinas,
Sudoku

Comparación de todas las
técnicas vistas



**Modalidad: Presencial
+ laboratorio**



**Traer: Laptop con
Python y Haskell
instalados**